MINOR PROJECT REPORT

SYNCHRONOUS FIFO RTL DESIGN

Submitted by: S.Gayathri chandana

Course: VLSI

Minor project:04

ABSTRACT

This project describes the RTL design and simulation of a single-clock synchronous First-In, First-Out (FIFO) buffer. The FIFO stores up to 16 data words, each 8 bits wide. Read and write operations use a common clock. Read and write pointers manage the data order, and an occupancy counter records the number of stored words. Status outputs indicate full, empty, almost-full, and almost-empty conditions. The design is simulated in ModelSim using Verilog testbenches.

1. INTRODUCTION

A FIFO is a memory structure in which the first data item written is the first item read. FIFOs are commonly used for temporary data buffering between digital design blocks. This project implements a synchronous FIFO in which both read and write operations use the same clock.

2. OBJECTIVES

• Design a synchronous FIFO with 16 storage locations and an 8-bit data width.

• Use read and write pointers to preserve FIFO order.

• Track stored words using an occupancy counter.

• Generate full, empty, almost_full, and almost_empty status flags.

• Simulate and verify FIFO behavior in ModelSim.

3. DESIGN SPECIFICATION

Parameter / Signal	Value	Description
FIFO type	Synchronous	Single common clock
Depth	16 entries	Stores up to 16 words
Data width	8 bits	Each stored word is 8 bits
Read/write pointers	4 bits each	Address 16 locations
Occupancy counter	5 bits	Represents values 0 through 16
full	occupancy = 16	No additional write accepted
empty	occupancy = 0	No valid word available to read
almost_full	occupancy >= 14	Two or fewer free entries
almost_empty	occupancy <= 1	One or zero words remain
Reset	Active-low	Check RTL for exact reset behavior

4. FUNCTIONAL OPERATION

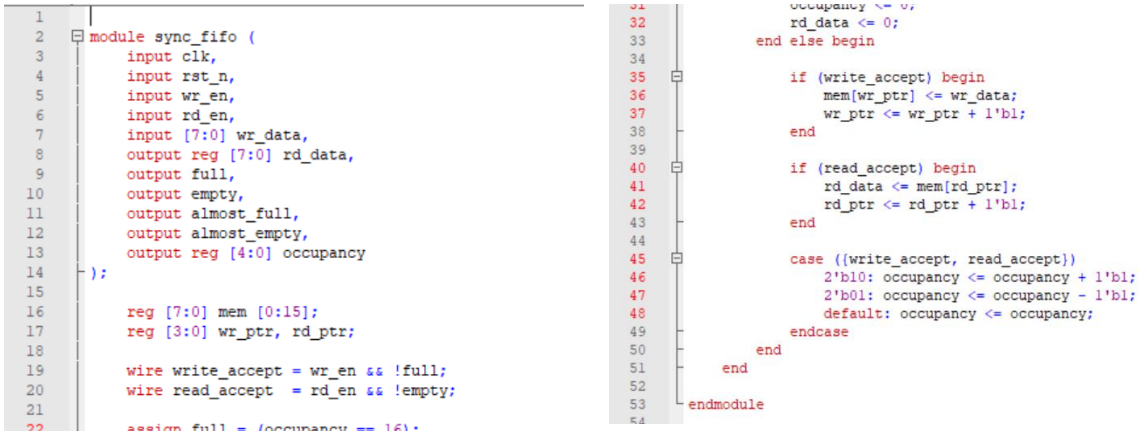
The FIFO consists of a 16 × 8 memory array, a read pointer, a write pointer, and an occupancy counter. A write is accepted when wr_en is high and full is low. A read is accepted when rd_en is high and empty is low. After an accepted write or read, the corresponding pointer advances. The occupancy counter increments for a write-only operation, decrements for a read-only operation, and remains unchanged when both or neither operation is accepted.

[Insert FIFO block diagram here]

Suggested labels: clk, rst_n, wr_en, wr_data, rd_en, rd_data, memory (16 × 8), read pointer, write pointer, occupancy, and status flags.

5. RTL IMPLEMENTATION

The RTL is written in Verilog. The memory contains 16 words of 8 bits each. Four-bit pointers address the memory, and a five-bit occupancy counter represents values from 0 to 16. Status flags are generated from the occupancy value.



```
module sync_fifo (
    input clk,
    input rst_n,
    input wr_en,
    input rd_en,
    input [7:0] wr_data,
    output reg [7:0] rd_data,
    output full,
    output empty,
    output almost_full,
    output almost_empty,
    output reg [4:0] occupancy
);

    reg [7:0] mem [0:15];
    reg [3:0] wr_ptr, rd_ptr;

    wire write_accept = wr_en && !full;
    wire read_accept  = rd_en && !empty;

    assign full = (occupancy == 16);
    assign empty = (occupancy == 0);
    assign almost_full = (occupancy >= 14);
    assign almost_empty = (occupancy <= 1);

    always @(posedge clk) begin
        if (!rst_n) begin
            wr_ptr <= 0;
            rd_ptr <= 0;
            occupancy <= 0;
            rd_data <= 0;
        end else begin

            if (write_accept) begin
                mem[wr_ptr] <= wr_data;
            rd_data <= 0;
        end else begin

            if (write_accept) begin
                mem[wr_ptr] <= wr_data;
                wr_ptr <= wr_ptr + 1'b1;
            end

            if (read_accept) begin
                rd_data <= mem[rd_ptr];
                rd_ptr <= rd_ptr + 1'b1;
            end

            case ({write_accept, read_accept})
                2'b10: occupancy <= occupancy + 1'b1;
                2'b01: occupancy <= occupancy - 1'b1;
                default: occupancy <= occupancy;
            endcase
        end
    end

endmodule
```

Fig: screenshot of source Verilog code

6. TESTBENCH AND SIMULATION PROCEDURE

1. Create a ModelSim project and add the RTL and testbench Verilog files.
2. Compile the files using Compile → Compile All.
3. Start simulation by selecting the required testbench module from the work library.
4. Add relevant signals to the Wave window and run the simulation.
5. Inspect data order, occupancy, and status flags in the waveform and Transcript window.

```
1
2   `timescale 1ns/1ps
3
4   module tb_sync_fifo;
5
6       reg clk, rst_n, wr_en, rd_en;
7       reg [7:0] wr_data;
8
9       wire [7:0] rd_data;
10      wire full, empty, almost_full, almost_empty;
11      wire [4:0] occupancy;
12
13      sync_fifo dut (
14          .clk(clk),
15          .rst_n(rst_n),
16          .wr_en(wr_en),
17          .rd_en(rd_en),
18          .wr_data(wr_data),
19          .rd_data(rd_data),
20          .full(full),
21          .empty(empty),
22          .almost_full(almost_full),
23          .almost_empty(almost_empty),
24          .occupancy(occupancy)
25      );
26
27      always #5 clk = ~clk;
28
29      initial begin
30          clk = 0;
31          rst_n = 0;
32          wr_en = 0;
33          rd_en = 0;
34          wr_data = 0;
35
36          #12;
```

```
33          rd_en = 0;
34          wr_data = 0;
35
36          #12;
37          rst_n = 1;
38
39          // Write three words
40          @(negedge clk);
41          wr_en = 1;
42          wr_data = 8'h11;
43
44          @(negedge clk);
45          wr_data = 8'h22;
46
47          @(negedge clk);
48          wr_data = 8'h33;
49
50          @(negedge clk);
51          wr_en = 0;
52          rd_en = 1;
53
54          // Read three words
55          repeat (3) @(negedge clk);
56          rd_en = 0;
57
58          #10;
59          $display("FIFO test completed");
60          $finish;
61      end
62
63  endmodule
```

Fig: screenshot of testbench source code

```
# Compile of sync_fifo.v was successful.
# Compile of tb_sync_fifo was successful.
```

Fig: screenshot of successful compilation / Transcript

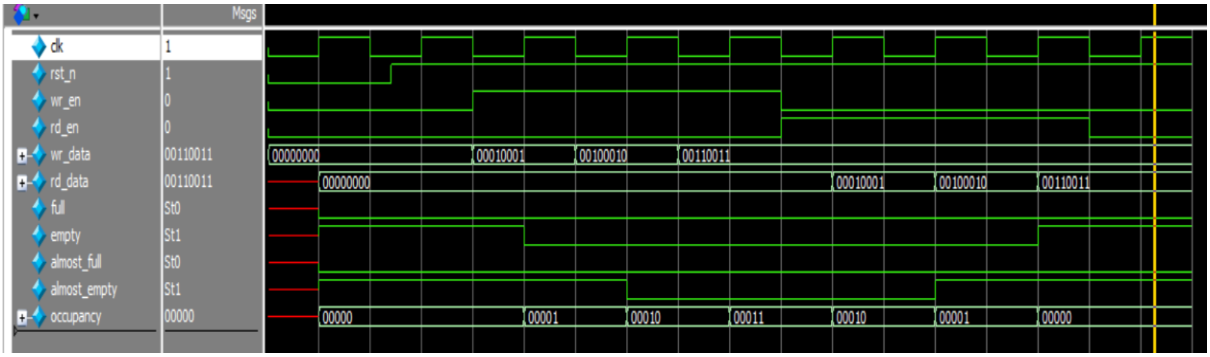


Fig: screenshot of basic read/write waveform

7. VERIFICATION AND RESULTS

The basic waveform showed three data words (0x11, 0x22, and 0x33) being written and read in FIFO order. A separate full-condition waveform showed occupancy reaching binary 10000 (decimal 16) and full asserted. The almost-empty testbench produced PASS messages at occupancy 1 and occupancy 0. Complete the evidence column below only after confirming the relevant Transcript output.

Test	Expected result	Evidence / status
Basic write/read	Data read in write order	Waveform: 0x11, 0x22, 0x33
Fill 16 entries	occupancy = 16, full = 1	Observed in waveform
Almost-empty at 1	almost_empty = 1	Transcript PASS observed
Empty at 0	empty = 1, almost_empty = 1	Transcript PASS observed
Extra write while full	Occupancy stays at 16	Add PASS screenshot after confirmation
Read while empty	Occupancy stays at 0	Add PASS screenshot after confirmation

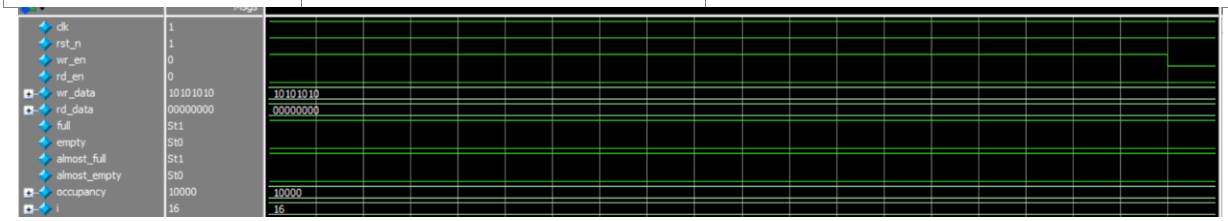


Fig:screenshot of full-flag waveform

```
# PASS: Almost-empty at occupancy 1
# PASS: FIFO empty at occupancy 0
# FIFO empty test completed
# ** Note: $finish    : D:/tb_fifo_empty.v(73)
#    Time: 66 ns  Iteration: 0  Instance: /tb_fifo_empty
# 1
# Break in Module tb_fifo_empty at D:/tb_fifo_empty.v line 73
```

Fig: screenshot of almost-empty Transcript PASS messages

```
# PASS: Read rejected when FIFO is empty
# Underflow test completed
# ** Note: $finish    : D:/tb_fifo_underflow.v(54)
#    Time: 36 ns  Iteration: 0  Instance: /tb_fifo_underflow
# 1
# Break in Module tb_fifo_underflow at D:/tb_fifo_underflow.v line 54
```

Fig: screenshot of underflow test PASS message

8. STATUS FLAG BEHAVIOR

Occupancy	empty	almost_empty	almost_full	full
0	1	1	0	0
1	0	1	0	0
13	0	0	0	0
14	0	0	1	0
15	0	0	1	0
16	0	0	1	1

9. CONCLUSION

A 16-entry, 8-bit synchronous FIFO was designed in Verilog and simulated in ModelSim. The basic simulation demonstrated FIFO ordering and occupancy changes. The full condition and almost-empty conditions were also observed. The project demonstrates how pointers, an occupancy counter, and status flags support temporary data storage. Before final submission, add the requested screenshots and confirm any remaining verification tests.